

JAVA 17 OCP · YDS STUDY SERIES

Unit 10 · Streams — Practice Quiz

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu belge Java 17/OCP odağında hazırlanmış **özgün çalışma soruları** içerir; gerçek sınav sorusu değildir. Önerilen süre 20–25 dakikadır. Her pipeline için source, intermediate operation ve terminal operation'ı işaretle.

Bu sette **8 özgün soru** vardır; her soruda aksi belirtilmedikçe tek doğru seçenek seçilir.

Sorular

Soru 1

Aşağıdaki kodun çıktısı nedir?

```
import java.util.stream.Stream;

public class LazyQuiz {
    public static void main(String[] args) {
        Stream.of("ant", "bear")
            .filter(s -> {
                System.out.print("F");
                return s.length() > 3;
            })
            .map(s -> {
                System.out.print("M");
                return s.toUpperCase();
            })
            .findFirst()
            .ifPresent(System.out::print);
    }
}
```

- A. FFMBEAR
- B. FMFFMBEAR
- C. FMBEAR
- D. Intermediate operation'lar lazy olduğu için hiçbir çıktı oluşmaz.

Soru 2

Aşağıdaki programın Java 17 reference JDK üzerindeki sonucu hangisidir?

```
import java.util.stream.Stream;

public class ReuseQuiz {
    public static void main(String[] args) {
        Stream<Integer> numbers = Stream.of(1, 2, 3);
        System.out.print(numbers.count());
        System.out.print(numbers.count());
    }
}
```

- A. 33 yazdırır.
- B. Kod derlenmez; aynı variable iki kez kullanılamaz.
- C. Önce 3 yazdırır, ardından ikinci terminal operation runtime'da `IllegalStateException` fırlatır.
- D. İlk `count()` stream'i otomatik olarak yeniden oluşturduğu için 6 yazdırır.

Soru 3

Stream davranışı hakkında doğru olan **iki seçeneği** seçin.

- A. Empty stream üzerinde `allMatch()` `true` döndürür.
- B. Empty stream üzerinde `anyMatch()` `true` döndürür.
- C. Intermediate operation, pipeline declaration edildiği anda bütün elementlere uygulanır.
- D. `Stream.iterate(0, n -> n + 1).limit(5).count()` sona erer ve 5 döndürür.
- E. Infinite stream üzerinde `sorted().findFirst()` her zaman kısa devre yapıp sona erer.

Soru 4

`fallback()` çağrıldığında `F` yazdırıyor ve `"other"` döndürüyor. Aşağıdaki programın sonucu hangisidir?

```
import java.util.Optional;

public class OptionalQuiz {
    static String fallback() {
        System.out.print("F");
        return "other";
    }

    public static void main(String[] args) {
        String result = Optional.of("ready").orElse(fallback());
        System.out.print(result);
    }
}
```

- A. Yalnız `ready` yazdırılır; present `Optional` fallback'i değerlendirmez.
- B. `Fready` yazdırılır; `orElse()` argument'ı eager değerlendirilir.
- C. `Fother` yazdırılır; fallback value her zaman seçilir.
- D. Kod derlenmez; `orElse()` yalnız empty `Optional` ile çağrılabilir.

Soru 5

`Collectors.groupingBy()` ve `Collectors.partitioningBy()` için doğru ifade hangisidir?

- A. `partitioningBy()` yalnız input'ta karşılaşılan Boolean key'i üretir.
- B. `groupingBy()` classifier sonucu ne olursa olsun yalnız `true` ve `false` key'lerini üretir.
- C. `partitioningBy()` sonucu, gruplardan biri boş olsa bile `true` ve `false` key'lerinin ikisini de içerir.
- D. İki collector her input için aynı key type ve aynı sayıda entry üretir.

Soru 6

Şu teknik cümlelerin doğal Türkçe çevirisi ve doğru çıkarımı hangisidir?

A short-circuiting terminal operation may avoid processing every element, but it does not guarantee that an arbitrary infinite pipeline will terminate.

- A. "Short-circuiting bir terminal operation her elementi işlemeyebilir; ancak bu, herhangi bir infinite pipeline'ın sona ereceğini garanti etmez." Operation sırası ve önceki stateful adımlar ayrıca incelenmelidir.
- B. "Short-circuiting terminal operation bütün infinite stream'leri hemen bitirir." Intermediate operation'ların etkisi yoktur.
- C. "Infinite stream hiçbir terminal operation ile sona eremez." `limit()` ve eşleşme operation'ları da işe yaramaz.
- D. "Terminal operation yalnız bütün elementler işlendikten sonra başlayabilir." Pipeline element-by-element çalışmaz.

Soru 7 · İki collector'ın bağımsız girdisi

Aşağıdaki tam Java 17 programı için doğru sonuç hangisidir? **Bir seçenek seçin.**

```
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class TeeingQuiz {
    public static void main(String[] args) {
        String result = Stream.of("ant", "bear", "cat")
            .collect(Collectors.teeing(
                Collectors.filtering(s -> s.length() > 3,
                    Collectors.toList()),
                Collectors.counting(),
                (kept, total) -> kept.size() + "/" + total));
        System.out.print(result);
    }
}
```

- A. 1/1
- B. 1/3
- C. 3/3
- D. Kod derlenmez; teeing Java 17'de yoktur.

Soru 8 · Optional.map ve flatMap null sonucu

Aşağıdaki tam Java 17 programı için doğru sonuç hangisidir? **Bir seçenek seçin.**

```
import java.util.Optional;

public class OptionalNullQuiz {
    public static void main(String[] args) {
        int value = Optional.of("x")
            .<Integer>map(s -> null).orElse(7);
        System.out.print(value + ":");
        Optional.of("x").flatMap(s -> null);
        System.out.print("end");
    }
}
```

- A. 7:end
- B. null:end
- C. Önce 7: yazar, sonra NullPointerException oluşur.
- D. Kod derlenmez; lambda null döndüremez.

Cevaplar ve açıklamalar

Soru 1 — A

- **A doğru:** `ant` yalnız filter'dan geçip `F` yazdırır. `bear` filter'da ikinci `F`yi, `map`'te `My`i üretir; `findFirst()` değeri bulur ve `BEAR` yazdırılır.
- **B yanlış:** Pipeline her element için bütün stage'leri koşulsuz çalıştırmaz; `ant` filter'dan geçemez.
- **C yanlış:** İlk element de filter tarafından değerlendirilir.
- **D yanlış:** Terminal operation olan `findFirst()` pipeline'ı tetikler.

Soru 2 — C

- **A yanlış:** Bir stream yalnız bir kez consume edilebilir.
- **B yanlış:** Bu kural compile-time'da variable kullanımını yasaklamaz.
- **C doğru:** Bu exact pipeline'da ilk `count()` 3 yazdırıp stream'i consume eder; Java 17 reference JDK ikinci `count()` çağrısında reuse'u tespit ederek runtime `IllegalStateException` üretir. Genel Stream API sözleşmesi, bütün olası reuse biçimlerinin mutlaka tespit edileceğini garanti etmez.
- **D yanlış:** Stream terminal operation sonrasında otomatik olarak yeniden oluşturulmaz.

Soru 3 — A ve D

- **A doğru:** Bu, vacuous truth (boş kümede doğruluk) davranışıdır.
- **B yanlış:** Empty stream'de eşleşen element yoktur; `anyMatch()` `false` döndürür.
- **C yanlış:** Intermediate operation'lar terminal operation çağrılana kadar lazy'dir.
- **D doğru:** `limit(5)` infinite source'u bu pipeline için beş elementle sınırlar; `count()` 5 döndürür.
- **E yanlış:** `sorted()` bütün input'u görmesi gereken stateful bir operation'dır; infinite source tamamlanmadığı için `findFirst()` e değer aktaramaz.

Soru 4 — B

- **A yanlış:** `orElse()` ye geçirilen method argument normal Java evaluation order gereği önce hesaplanır.
- **B doğru:** `fallback()` `F` yazdırır; `Optional` present olduğu için seçilen sonuç yine `"ready"` dir.
- **C yanlış:** Fallback hesaplanır ama present value yerine seçilmez.
- **D yanlış:** `orElse()` hem present hem empty `Optional` üzerinde çağrılabilir.

Lazy fallback gerekirse `orElseGet(() -> fallback())` kullanılır; present durumda supplier çağrılmaz.

Soru 5 — C

- **A yanlış:** `partitioningBy()` iki Boolean key'i de üretir.
- **B yanlış:** `groupingBy()` classifier'ın gerçekten ürettiği key'lerle çalışır ve key type Boolean olmak zorunda değildir.
- **C doğru:** Bir partition boş olsa da map'te `true` ve `false` entry'leri bulunur.
- **D yanlış:** Collector'ların key contract'ları ve entry üretme davranışları farklıdır.

Soru 6 — A

- **A doğru:** `may` olasılık, `does not guarantee` ise garanti yokluğu bildirir. Örneğin infinite source'tan önce sonuç isteyen `sorted()` sona ermeyi engelleyebilir.
- **B yanlış:** Stateful intermediate operation veya hiç eşleşmeyen predicate, termination'ı engelleyebilir.
- **C yanlış:** Uygun `limit()`, `findFirst()` veya eşleşme koşulu bazı infinite pipeline'ları sonlandırabilir.
- **D yanlış:** Stream pipeline çoğunlukla elementleri stage'ler boyunca demand-driven işler.

Soru 7 — B

Başarıyla derlenir ve `1/3` yazar. İki downstream collector aynı üç öğeyi alır; filtering yalnız ilk collector'ın tuttuğu listeyi daraltır. A filtreden çıkanları ikinci collector'ın da kaybettiğini varsayar; C filtreyi yok sayar. D yanlıştır: teeing Java 17'de mevcuttur. Pipeline'a collect öncesinde filter eklenseydi her iki collector'ın girdisi daralırdı.

Soru 8 — C

Kod derlenir. `map` null sonucu `Optional.empty()` olarak ele alır; `orElse` bu yüzden 7 verir. `flatMap` ise mapper'ın `Optional` döndürmesini bekler; null `Optional` sonucu `NullPointerException` oluşturur. A iki API'nin null davranışını eşit sayar; B `orElse`'i yok sayar; D yanlıştır çünkü null reference dönüşü derleme açısından uygundur.